

Tips to build the Forecasting

Objectives

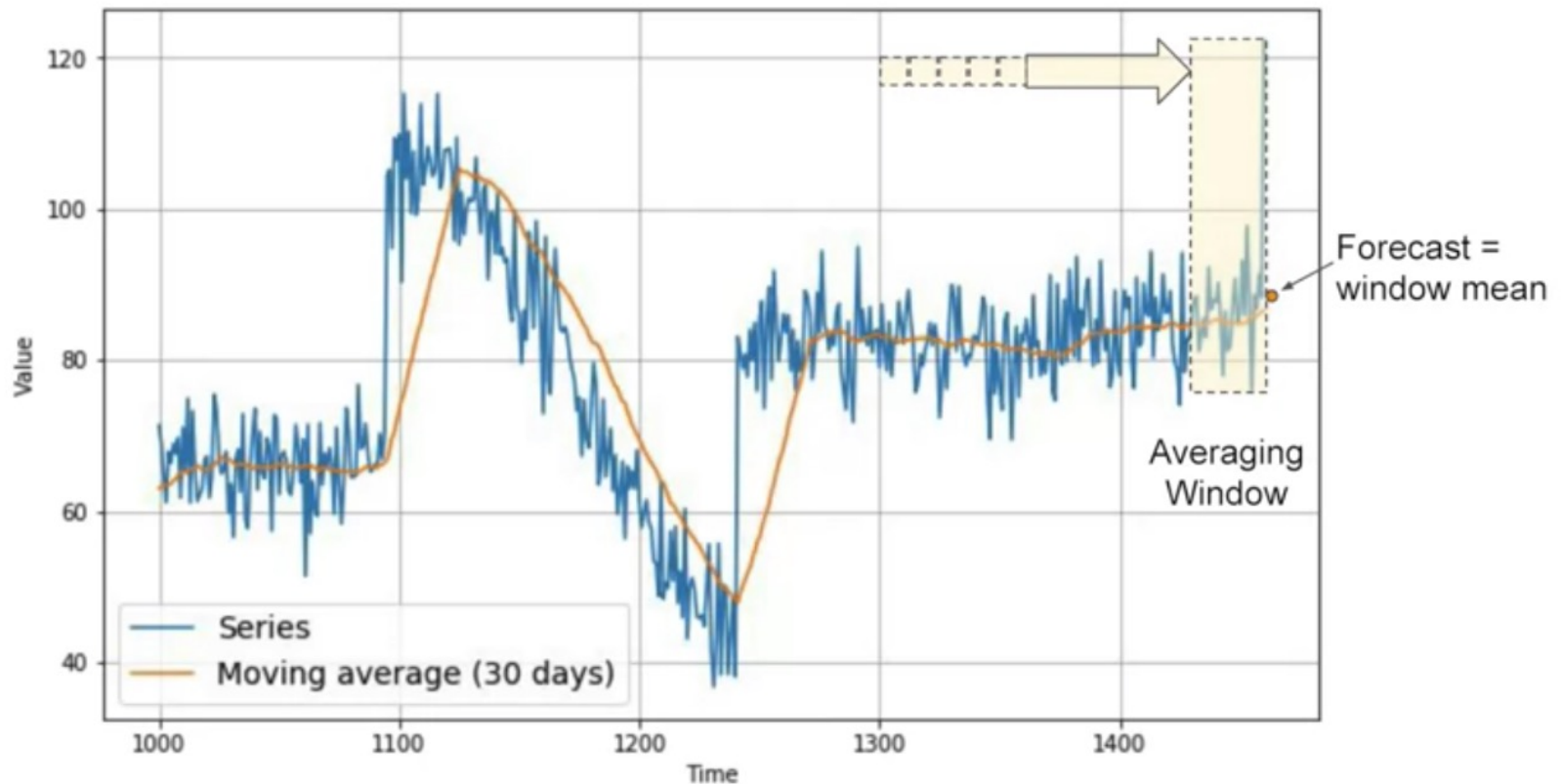
- ❑ Understand methods for Forecasting
- ❑ Implement Statistical Forecasting methods

Moving Average Methods

- ❑ The moving average method is a simple and widely used technique in time series forecasting.
- ❑ It's particularly useful for smoothing out short-term fluctuations and highlighting longer-term trends or cycles in the data.
- ❑ The basic idea behind the moving average method is to calculate the average of a set of consecutive data points within a moving window and use this average as a forecast for future values.

Moving Average Methods

Moving Average



Moving Average Methods

Simple Moving Average (SMA)

- The Simple Moving Average is calculated by taking the average of a specified number of consecutive data points, known as the window or period.
- Formula
$$SMA_t = \frac{X_{t-1} + X_{t-2} + \dots + X_{t-n}}{n}$$
- SMA_t is the moving average at time t , $X_{t-1}, X_{t-2}, \dots, X_{t-n}$ are the data points within the window, and n is the number of data points in the window.

Moving Average Methods

Weighted Moving Average (WMA)

- The Weighted Moving Average assigns different weights to different data points within the window, giving more importance to recent observations.

- Formula

$$WMA_t = \frac{w_1 \cdot X_{t-1} + w_2 \cdot X_{t-2} + \dots + w_n \cdot X_{t-n}}{w_1 + w_2 + \dots + w_n}$$

- WMA_t is the weighted moving average at time t , $X_{t-1}, X_{t-2}, \dots, X_{t-n}$ are the data points within the window, and $w_1, w_2, \dots, w_n$ are the corresponding weights.

Moving Average Methods

Exponential Moving Average (EMA):

- The Exponential Moving Average places more weight on recent observations and less weight on older observations, exponentially decreasing the weights as you move back in time.
- Formula:

$$EMA_t = \alpha \cdot X_t + (1 - \alpha) \cdot EMA_{t-1}$$

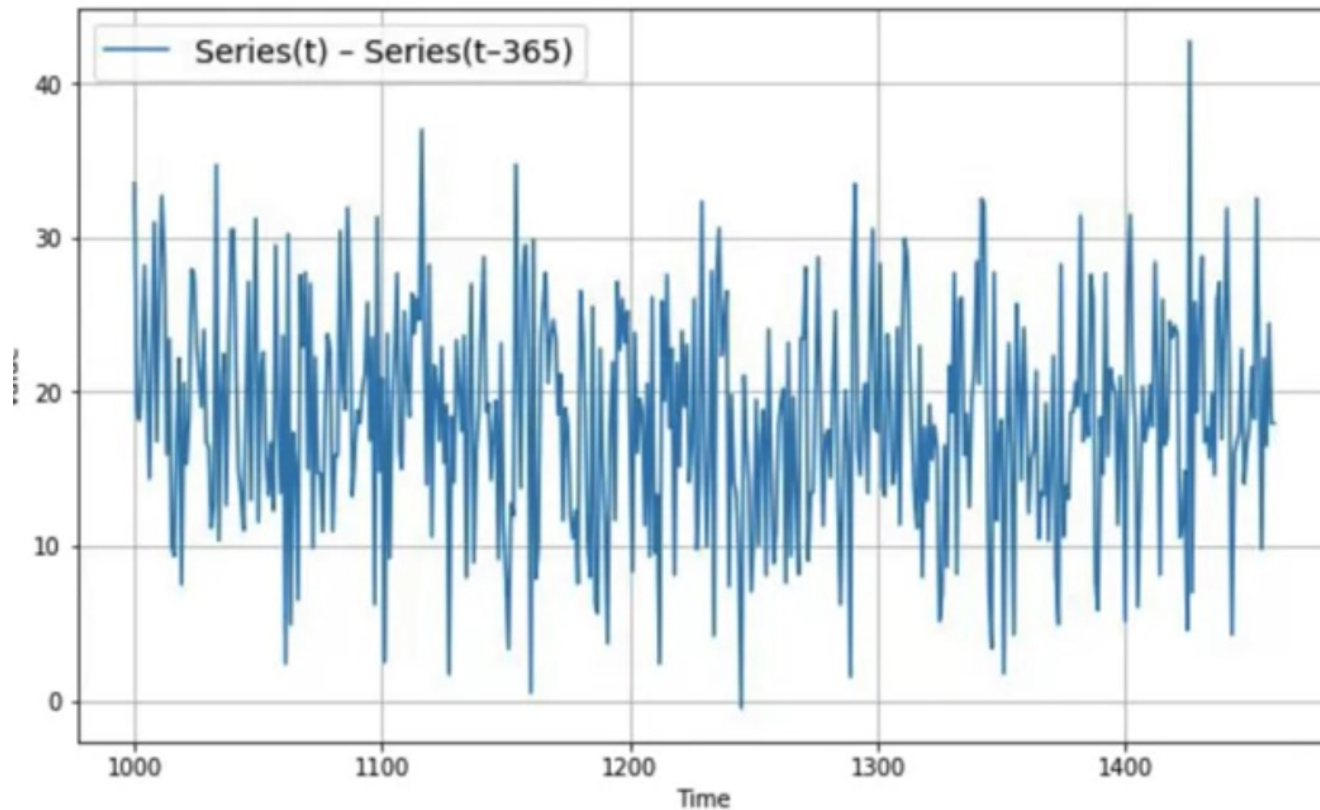
- EMA_t is the exponential moving average at time t , X_t is the actual value at time t , EMA_{t-1} is the EMA at the previous time step, and α is the smoothing factor ($0 < \alpha < 1$)

Differencing methods

- ❑ Differencing is a common technique used in time series forecasting to make a series stationary.
- ❑ Stationarity is an important concept in time series analysis, and differencing helps stabilize the mean and variance of a series, making it more amenable to modeling.
- ❑ The differencing method involves computing the difference between consecutive observations, aiming to remove trends and seasonality from the data.

Differencing methods

Differencing



Differencing methods

First-Order Differencing:

- In first-order differencing, the value at each time step is subtracted from the value at the previous time step. This helps eliminate a linear trend in the data.

- Formula
$$Y'_t = Y_t - Y_{t-1}$$

- Y'_t is the differenced series at time t , Y_t is the original series at time t , and Y_{t-1} is the original series at the previous time step.

Differencing methods

Seasonal Differencing:

- Seasonal differencing is used when a series exhibits periodic patterns or seasonality. It involves subtracting the value at each time step from the value at the same time step in the previous season.

- Formula

$$Y'_t = Y_t - Y_{t-n}$$

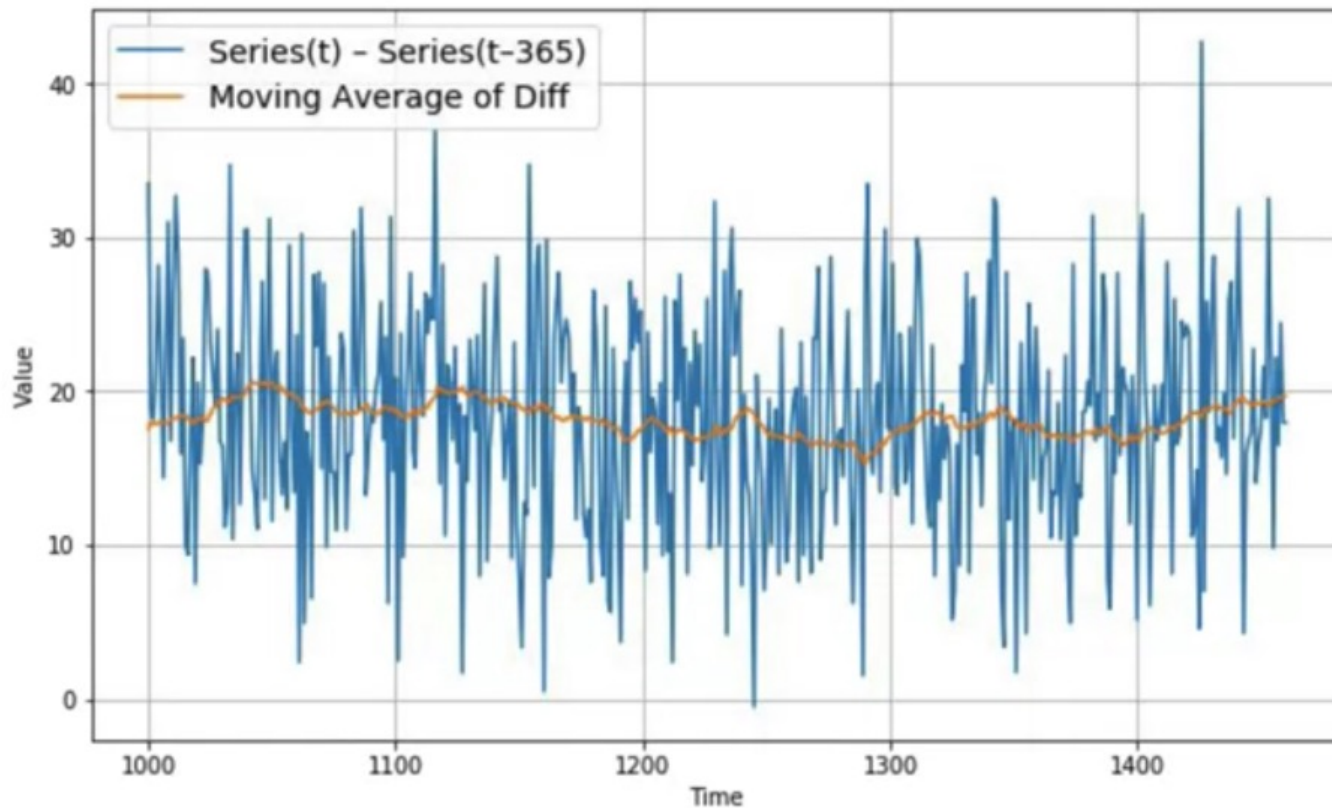
- Y'_t is the seasonally differenced series at time t , Y_t is the original series at time t , and Y_{t-n} is the original series at the same time step in the previous season, where n is the seasonal period

Differencing methods

- Differencing has several benefits in time series forecasting:
 - Stationarity: Differencing can help make a time series stationary by removing trends and seasonality. Stationary series are often easier to model and can lead to more accurate forecasts.
 - Stabilizing Variance: Differencing can stabilize the variance of a time series, making it more constant over time.
 - Removing Trends: By subtracting consecutive values, differencing can remove linear trends and make the series more amenable to modeling with models that assume stationarity.

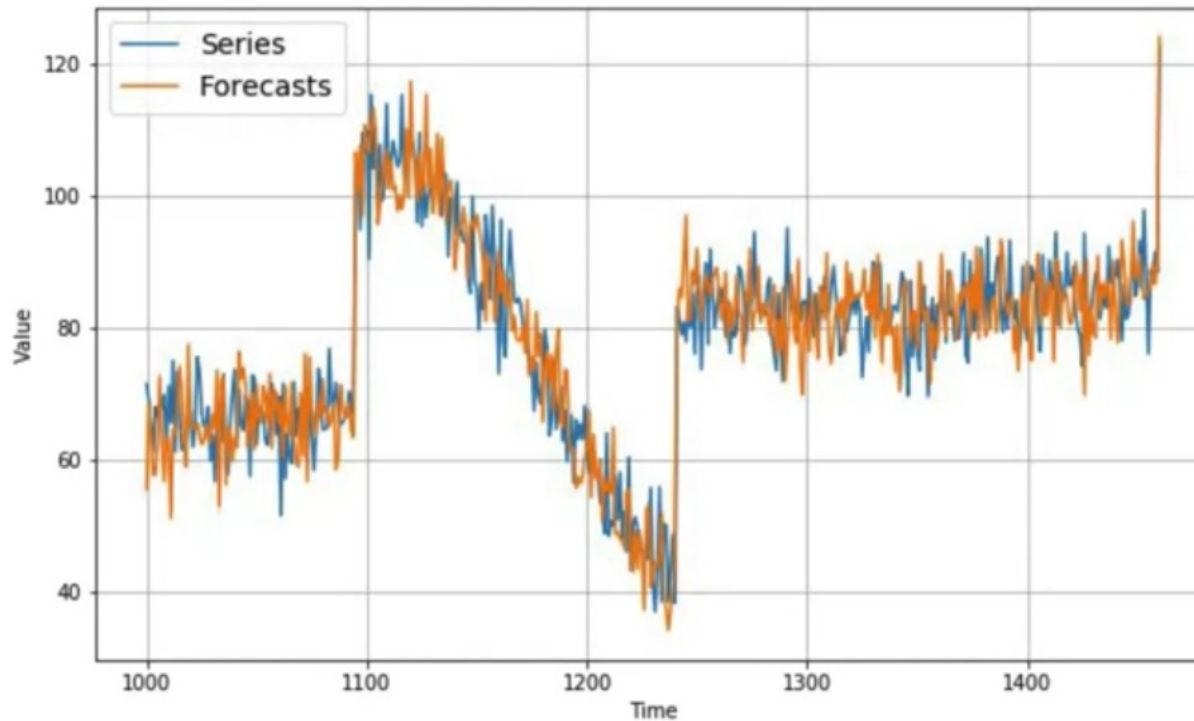
Moving Average on Differenced Time Series

Moving Average on Differenced Time Series



Restoring the Trend and Seasonality

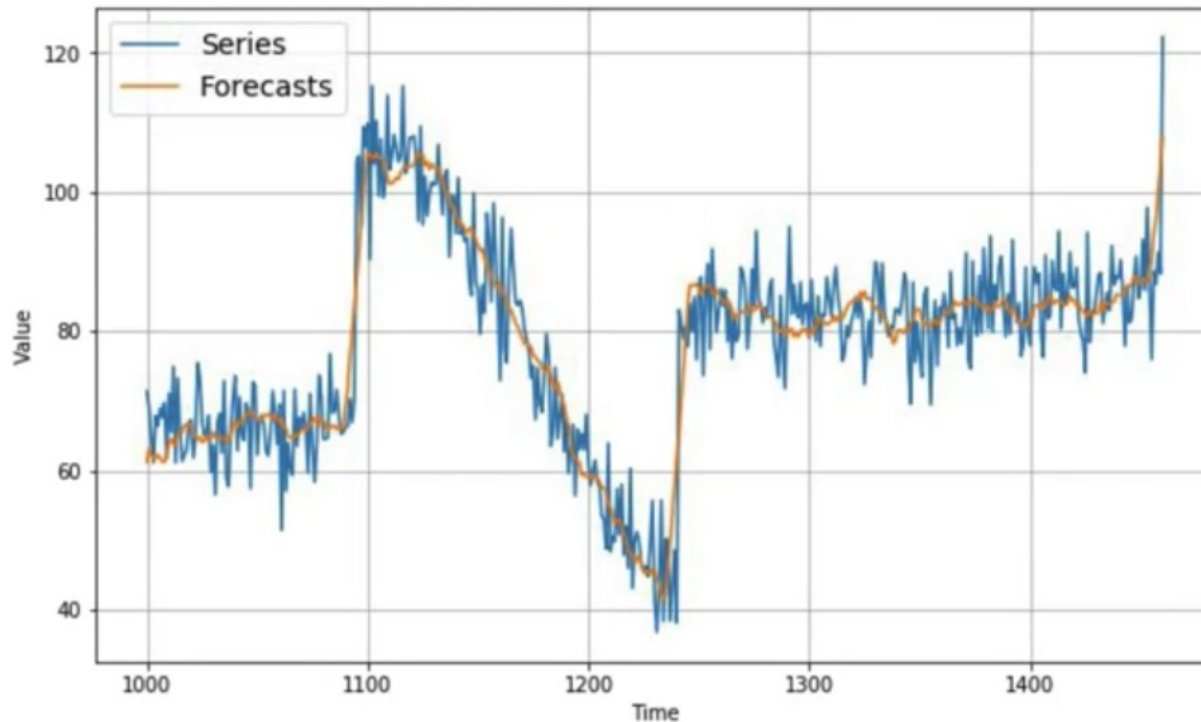
Restoring the Trend and Seasonality



Forecasts = moving average of differenced series + series($t - 365$)

Smoothing Both Past and Present Values

Smoothing Both Past and Present Values



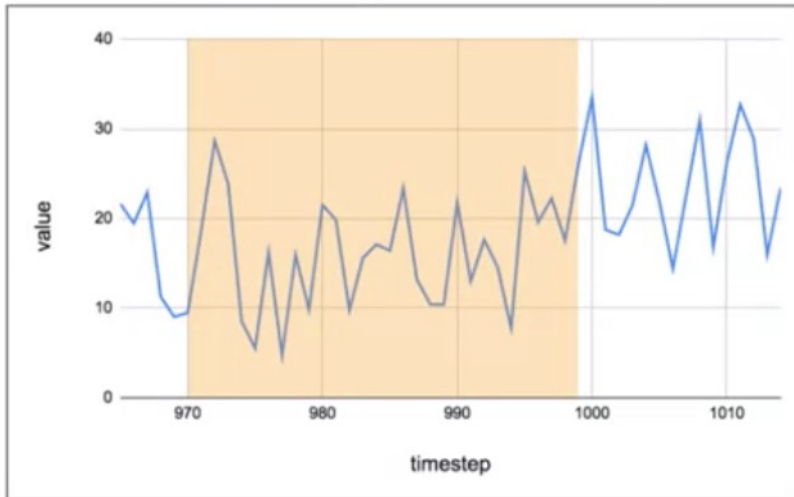
Forecasts = trailing moving average of differenced series + centered moving average of past series ($t - 365$)

Trailing versus centered windows

- ❑ Trailing window when computing the moving average of present values But we used a centered window to compute the moving average of past values.
- ❑ Moving averages using centered windows can be more accurate than using trailing windows.
- ❑ But we can't use centered windows to smooth present values since we don't know future values. However, to smooth past values we can afford to use centered windows.

Trailing versus centered windows

Trailing Moving Average of Differenced Series
(zoomed at t_{1000} , window size = 30)

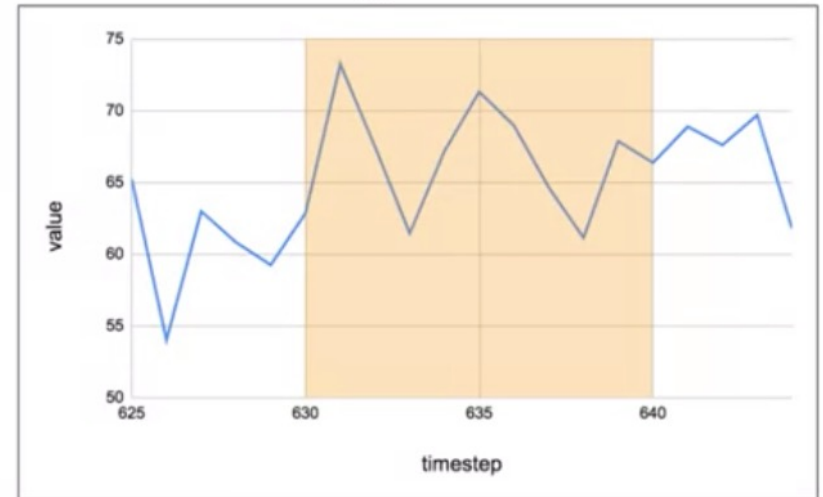


$$TMA_{t_{1000}} = (v_{t_{970}} + v_{t_{971}} + v_{t_{972}} + \dots + v_{t_{999}}) / 30$$



$$\text{forecast at } t_{1000} = TMA_{t_{1000}} + CMA_{t_{635}}$$

Centered Moving Average of Past Series ($t - 365$)
(zoomed at t_{635} , window size = 11)



$$CMA_{t_{635}} = (v_{t_{630}} + v_{t_{631}} + v_{t_{632}} + \dots + v_{t_{640}}) / 11$$



Statistical Forecasting

- Imports libs

```
[1] 1 import tensorflow as tf
     2 import numpy as np
     3 import matplotlib.pyplot as plt
```

Statistical Forecasting

□ Create Utilities

Visualizes time series data

Args:

time (array of int) - contains the time steps
 series (array of int) - contains the measurements
 for each time step
 format - line style when plotting the graph
 label - tag for the line
 start - first time step to plot
 end - last time step to plot

```

1 def plot_series(time, series, format="-", start=0, end=None):
2     # Setup dimensions of the graph figure
3     plt.figure(figsize=(10, 6))
4
5     if type(series) is tuple:
6
7         for series_num in series:
8             # Plot the time series data
9             plt.plot(time[start:end], series_num[start:end], format)
10
11     else:
12         # Plot the time series data
13         plt.plot(time[start:end], series[start:end], format)
14
15     # Label the x-axis
16     plt.xlabel("Time")
17
18     # Label the y-axis
19     plt.ylabel("Value")
20
21     # Overlay a grid on the graph
22     plt.grid(True)
23
24     # Draw the graph on screen
25     plt.show()
    
```

Statistical Forecasting

□ Create Utilities

```
1 def trend(time, slope=0):
2     # Compute the linear series given the slope
3     series = slope * time
4
5     return series
6
7 def seasonal_pattern(season_time):
8     # Generate the values using an arbitrary pattern
9     data_pattern = np.where(season_time < 0.4,
10                             np.cos(season_time * 2 * np.pi),
11                             1 / np.exp(3 * season_time))
12
13     return data_pattern
14
```

an arbitrary pattern

Args:

season_time (array of float) -
contains the measurements per time step

Returns:

data_pattern (array of float) -
contains revised measurement values according
to the defined pattern

Statistical Forecasting

□ Create Utilities

```
26 def seasonality(time, period, amplitude=1, phase=0):  
27     # Define the measured values per period  
28     season_time = ((time + phase) % period) / period  
29  
30     # Generates the seasonal data scaled by the defined amplitude  
31     data_pattern = amplitude * seasonal_pattern(season_time)  
32  
33     return data_pattern  
34
```

Repeats the same pattern at each period

Args:

time (array of int) - contains the time steps
period (int) - number of time steps before the pattern repeats
amplitude (int) - peak measured value in a period
phase (int) - number of time steps to shift the measured values

Returns:

data_pattern (array of float) - seasonal data scaled by the defined amplitude

Statistical Forecasting

□ Create Utilities

```
35 def noise(time, noise_level=1, seed=None):  
36     # Initialize the random number generator  
37     rnd = np.random.RandomState(seed)  
38  
39     # Generate a random number for each time  
40     # step and scale by the noise level  
41     noise = rnd.randn(len(time)) * noise_level  
42  
43     return noise
```

Generates a normally distributed noisy signal

Args:

time (array of int) - contains the time steps
noise_level (float) - scaling factor for the generated signal
seed (int) - number generator seed for repeatability

Returns:

noise (array of float) - the noisy signal

Statistical Forecasting

□ Generate the synthetic data

```
1 # Parameters
2 time = np.arange(4 * 365 + 1, dtype="float32")
3 baseline = 10
4 amplitude = 40
5 slope = 0.05
6 noise_level = 5
7
8 # Create the series
9 series = baseline + trend(time, slope) + seasonality(time, period=365, amplitude=amplitude)
10
11 # Update with noise
12 series += noise(time, noise_level, seed=42)
13
14 # Plot the results
15 plot_series(time, series)
```

Statistical Forecasting

□ Split the Dataset

```
1 # Define the split time
2 split_time = 1000
3
4 # Get the train set
5 time_train = time[:split_time]
6 x_train = series[:split_time]
7
8 # Get the validation set
9 time_valid = time[split_time:]
10 x_valid = series[split_time:]
```


Statistical Forecasting

□ Naive Forecast

```
1 # Generate the naive forecast
2 naive_forecast = series[split_time - 1:-1]
3
4 # Define time step
5 time_step = 100
6
7 # Print values
8 print(f'ground truth at time step {time_step}: {x_valid[time_step]}')
9 print(f'prediction at time step {time_step + 1}: {naive_forecast[time_step + 1]}')
10
```

```
1 # Plot the results
2 plot_series(time_valid, (x_valid, naive_forecast))
```

```
1 # Zooming in
2 plot_series(time_valid, (x_valid, naive_forecast), start=0, end=150)
```

Statistical Forecasting

□ Moving Average

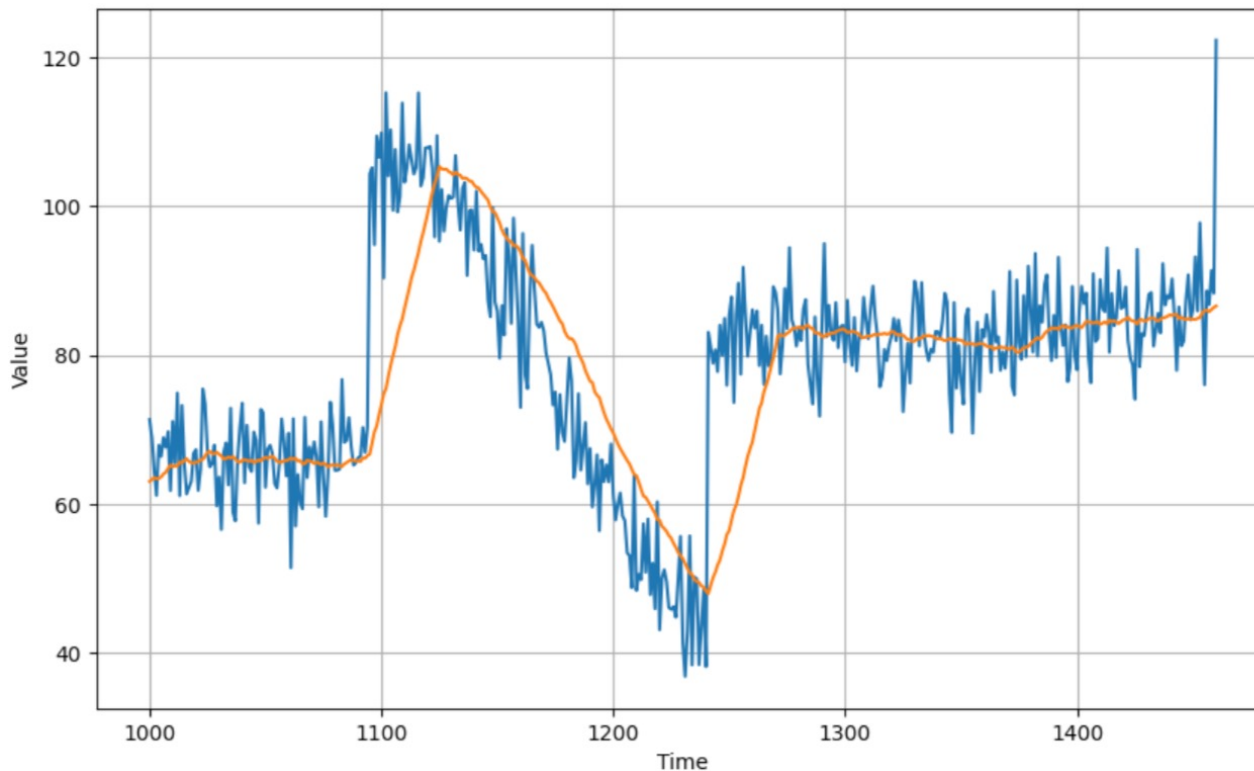
```
1 def moving_average_forecast(series, window_size):  
2     # Initialize a list  
3     forecast = []  
4  
5     # Compute the moving average based on the window size  
6     for time in range(len(series) - window_size):  
7         forecast.append(series[time:time + window_size].mean())  
8  
9     # Convert to a numpy array  
10    forecast = np.array(forecast)  
11  
12    return forecast
```

```
1 # Generate the moving average forecast  
2 moving_avg = moving_average_forecast(series, 30)[split_time - 30:]  
3  
4 # Plot the results  
5 plot_series(time_valid, (x_valid, moving_avg))
```

```
1 # Compute the metrics  
2 print(tf.keras.metrics.mean_squared_error(x_valid, moving_avg).numpy())  
3 print(tf.keras.metrics.mean_absolute_error(x_valid, moving_avg).numpy())
```

Statistical Forecasting

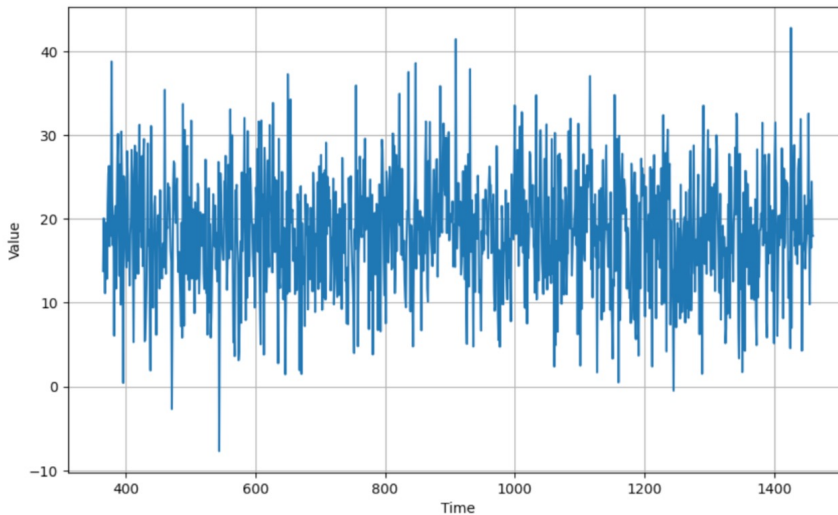
- Moving Average



Statistical Forecasting

□ Differencing

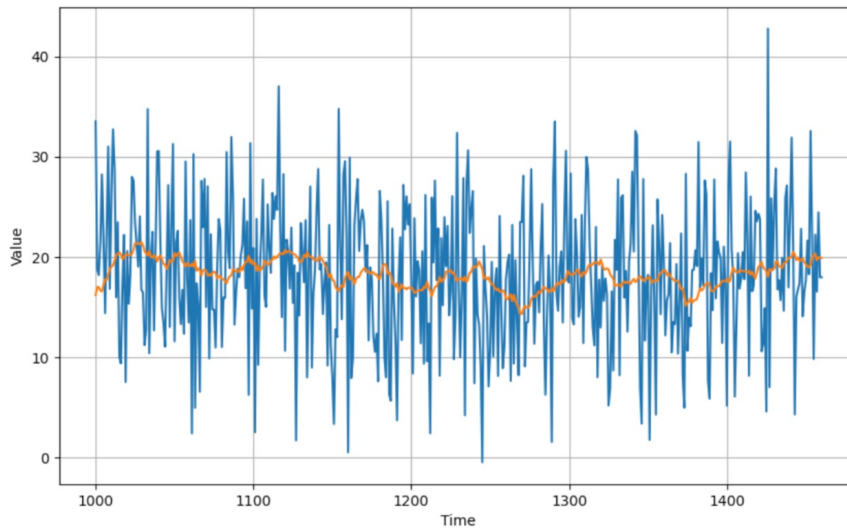
```
1 # Subtract the values at t-365 from original series
2 diff_series = (series[365:] - series[:-365])
3
4 # Truncate the first 365 time steps
5 diff_time = time[365:]
6
7 # Plot the results
8 plot_series(diff_time, diff_series)
```



Statistical Forecasting

□ Differencing

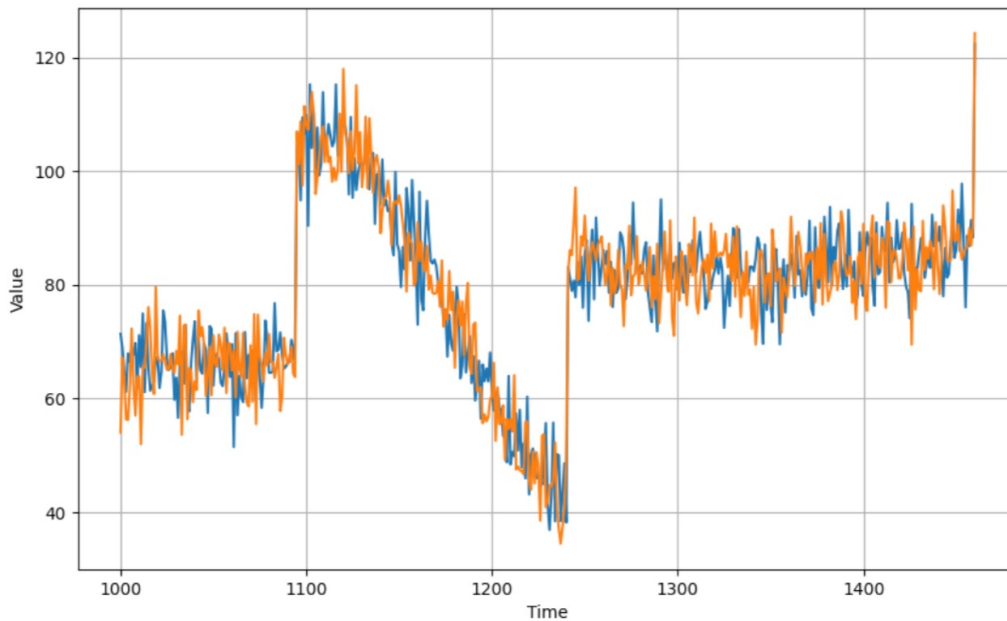
```
1 # Generate moving average from the time differenced dataset
2 diff_moving_avg = moving_average_forecast(diff_series, 30)
3
4 # Slice the prediction points that corresponds to the validation set time steps
5 diff_moving_avg = diff_moving_avg[split_time - 365 - 30:]
6
7 # Slice the ground truth points that corresponds to the validation set time steps
8 diff_series = diff_series[split_time - 365:]
9
10 # Plot the results
11 plot_series(time_valid, (diff_series, diff_moving_avg))
```



Statistical Forecasting

□ Differencing

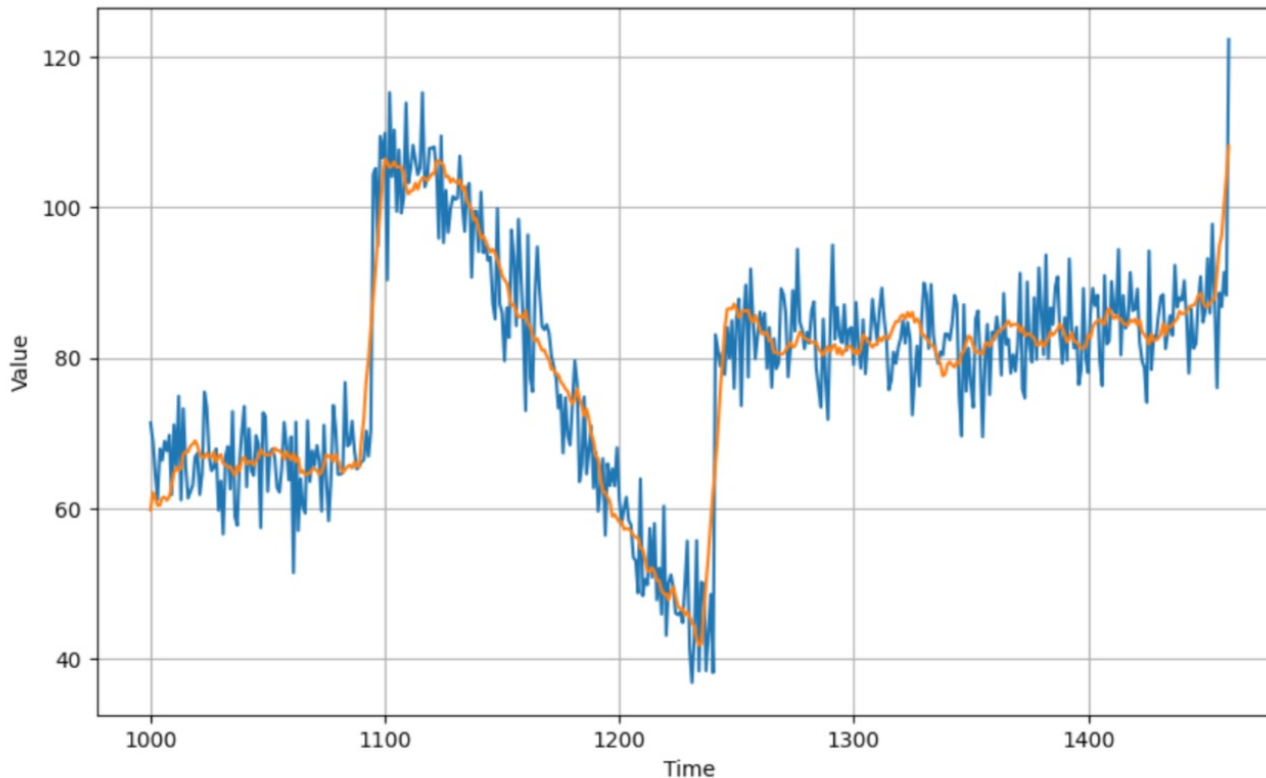
```
1 # Add the trend and seasonality from the original series
2 diff_moving_avg_plus_past = series[split_time - 365:-365] + diff_moving_avg
3
4 # Plot the results
5 plot_series(time_valid, (x_valid, diff_moving_avg_plus_past))
```



Statistical Forecasting

□ Smoothing

```
1 # Smooth the original series before adding the time differenced moving average
2 diff_moving_avg_plus_smooth_past = moving_average_forecast(series[split_time - 370:-359], 11) + diff_moving_avg
3
4 # Plot the results
5 plot_series(time_valid, (x_valid, diff_moving_avg_plus_smooth_past))
```



Statistical Forecasting

- Group discuss and give opinions about statistical forecasting methods

Summary

- ❑ Understand methods for Forecasting
- ❑ Implement Statistical Forecasting methods

Q & A